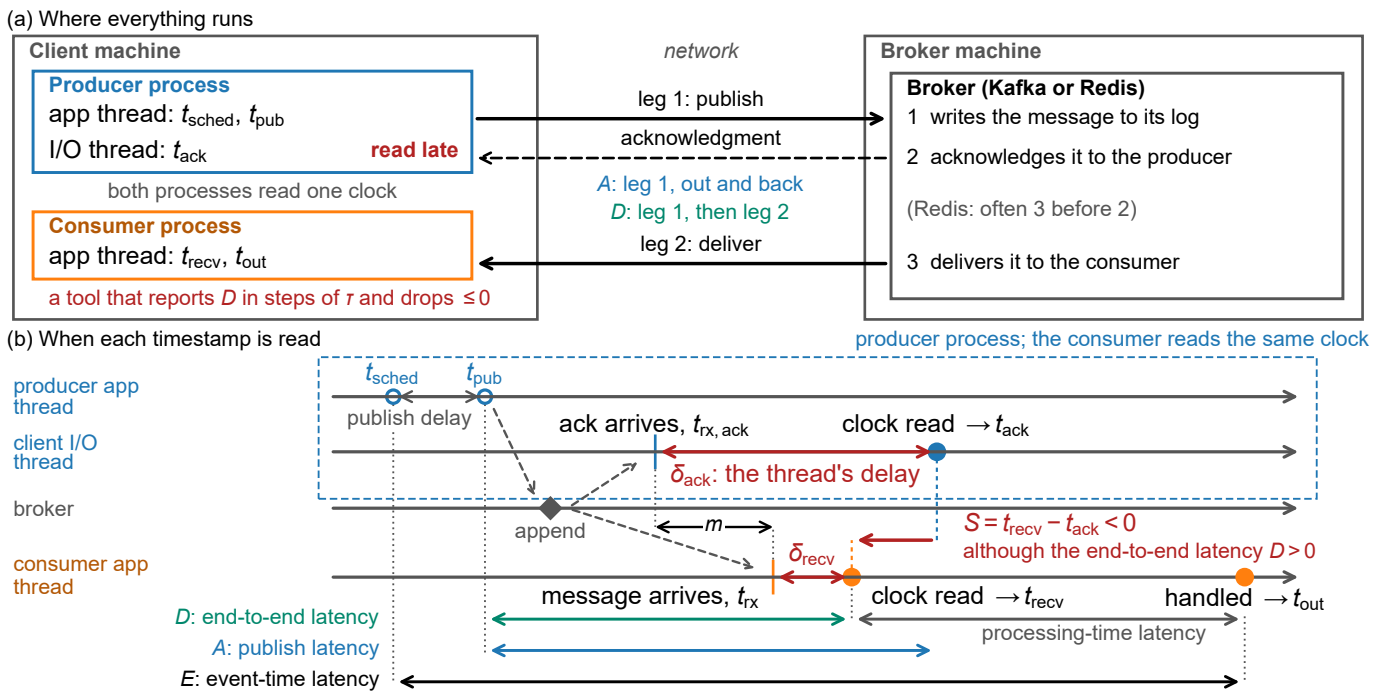

(a) Where everything runs
Client machine
Producer process
app thread: t_{sched}, t_{pub}
I/O thread: t_{ack}
read late
both processes read one clock
Consumer process
app thread: t_{recv}, t_{out}
a tool that reports D in steps of r and drops ≤ 0
network
leg 1: publish
acknowledgment
A: leg 1, out and back
D: leg 1, then leg 2
leg 2: deliver
Broker machine
Broker (Kafka or Redis)
1 writes the message to its log
2 acknowledges it to the producer
(Redis: often 3 before 2)
3 delivers it to the consumer
(b) When each timestamp is read
producer process; the consumer reads the same clock
producer app thread
client I/O thread
broker
consumer app thread
t_{sched}
t_{pub}
publish delay
ack arrives, $t_{rx,ack}$
clock read → t_{ack}
δ_{ack}: the thread's delay
append
m
$S = t_{recv} - t_{ack} < 0$
although the end-to-end latency $D > 0$
δ_{recv}
message arrives, t_{rx}
clock read → t_{recv}
handled → t_{out}
processing-time latency
D: end-to-end latency
A: publish latency
E: event-time latency